TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO BÀI LAB 03

SQLLITE

Môn học: NT118.R12 - Phát triển ứng dụng trên thiết bị di động.

Giảng viên hướng dẫn: TS. Phan Xuân Thiện.

Sinh viên thực hiện: Võ Minh Quân – 24521459.

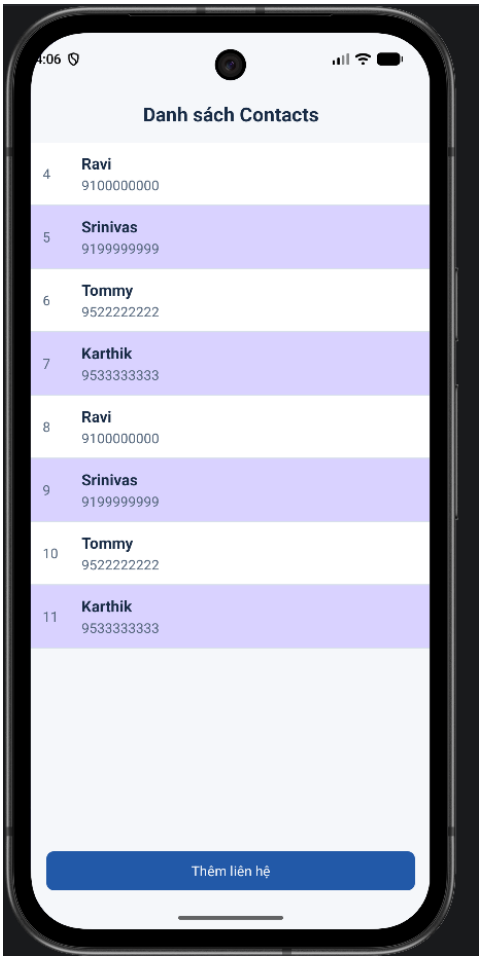
Tháng 10 năm 2026

Mục lục

Bài 1: Quản lý Contacts 3
Bài 2: Quản lý Students 7
PHỤ LỤC 13

Bài 1: Quản lý Contacts

Kết quả:



Định nghĩa lớp Contact:

```
public final class Contact {
    2 usages
    private final int id;
    2 usages
    private final String name;
    2 usages
    private final String phoneNumber;

    12 usages
    public Contact(String name, String phoneNumber) { this( id: 0, name, phoneNumber); }
    5 usages
    public Contact(int id, String name, String phoneNumber) {
        this.id = id;
        this.name = name;
        this.phoneNumber = phoneNumber;
    }
    4 usages
    public int getId() { return id; }
    public String getName() { return name; }
    5 usages
    public String getPhoneNumber() { return phoneNumber; }
}
```

File DatabaseHandler

```java
public final class DatabaseHandler extends SQLiteOpenHelper {
    1 usage
    public static final String DATABASE_NAME = "contactsManager";
    1 usage
    private static final int DATABASE_VERSION = 3;
    7 usages
    private static final String TABLE = "contacts";

    1 usage
    public DatabaseHandler(Context context) { this(context, DATABASE_NAME); }

    4 usages
    DatabaseHandler(Context context, String databaseName) {
        super(context.getApplicationContext(), databaseName, factory: null, DATABASE_VERSION);
    }

    @Override public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE contacts (id INTEGER PRIMARY KEY AUTOINCREMENT, "
                + "name TEXT NOT NULL, phone_number TEXT NOT NULL)");
        insertInitialContacts(db);
    }

    11 usages
    @Override public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        if (oldVersion < 2) {
            db.delete(TABLE, whereClause: "name = ? AND phone_number = ?",
                    new String[]{"Liên hệ mẫu 01", "0900000001"});
            db.delete(TABLE, whereClause: "name = ? AND phone_number = ?",
                    new String[]{"Liên hệ mẫu 02", "0900000002"});
        }
        if (oldVersion < 3) insertInitialContacts(db);
```

```
private void insertInitialContacts(SQLiteDatabase db) {
    insert(db, new Contact( name: "Ravi",  phoneNumber: "9100000000"));
    insert(db, new Contact( name: "Srinivas",  phoneNumber: "9199999999"));
    insert(db, new Contact( name: "Tommy",  phoneNumber: "9522222222"));
    insert(db, new Contact( name: "Karthik",  phoneNumber: "9533333333"));
    insert(db, new Contact( name: "Ravi",  phoneNumber: "9100000000"));
    insert(db, new Contact( name: "Srinivas",  phoneNumber: "9199999999"));
    insert(db, new Contact( name: "Tommy",  phoneNumber: "9522222222"));
    insert(db, new Contact( name: "Karthik",  phoneNumber: "9533333333"));
}

2 usages
private ContentValues values(Contact contact) {
    ContentValues values = new ContentValues();
    values.put("name", contact.getName());
    values.put("phone_number", contact.getPhoneNumber());
    return values;
}

9 usages
private long insert(SQLiteDatabase db, Contact contact) {
    return db.insertOrThrow(TABLE,  nullColumnHack: null, values(contact));
}

4 usages
public long addContact(Contact contact) { return insert(getWritableDatabase(), contact); }

7 usages
public Contact getContact(int id) {
    try (Cursor cursor = getReadableDatabase().query(TABLE,  columns: null,  selection: "id = ?",
            new String[]{String.valueOf(id)},  groupBy: null,  having: null,  orderBy: null)) {
        return cursor.moveToFirst() ? readContact(cursor) : null;
    }
}
```

```
public List<Contact> getAllContacts() {
    List<Contact> contacts = new ArrayList<>();
    try (Cursor cursor = getReadableDatabase().query(TABLE,  columns: null,  selection: null,  selectionArgs: null,
            groupBy: null,  having: null,  orderBy: "id ASC")) {
        while (cursor.moveToNext()) contacts.add(readContact(cursor));
    }
    return contacts;
}

3 usages
public int updateContact(Contact contact) {
    return getWritableDatabase().update(TABLE, values(contact),  whereClause: "id = ?",
            new String[]{String.valueOf(contact.getId())});
}

3 usages
public int deleteContact(Contact contact) {
    return getWritableDatabase().delete(TABLE,  whereClause: "id = ?",
            new String[]{String.valueOf(contact.getId())});
}

2 usages
private Contact readContact(Cursor cursor) {
    return new Contact(cursor.getInt(cursor.getColumnIndexOrThrow( columnName: "id")),
            cursor.getString(cursor.getColumnIndexOrThrow( columnName: "name")),
            cursor.getString(cursor.getColumnIndexOrThrow( columnName: "phone_number")));
}
```

File MainActivity.

```
public class MainActivity extends AppCompatActivity {
    4 usages
    private DatabaseHandler database;
    4 usages
    private DatabaseTasks tasks;
    4 usages
    private ContactAdapter adapter;

    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View view, WindowInsetsCompat insets) -> {
            Insets bars = insets.getInsets( typeMask: WindowInsetsCompat.Type.systemBars()
                    | WindowInsetsCompat.Type.displayCutout());
            view.setPadding(bars.left, bars.top, bars.right, bars.bottom);
            return insets;
        });

        database = new DatabaseHandler( context: this);
        tasks = new DatabaseTasks( context: this, database);
        adapter = new ContactAdapter( context: this);
        ListView list = findViewById(R.id.contactList);
        list.setAdapter(adapter);
        list.setEmptyView(findViewById(R.id.emptyView));
        list.setOnItemLongClickListener(( AdapterView<?> parent, View view, int position, long id) -> {
            Contact contact = adapter.getItem(position);
            tasks.submit(() -> database.deleteContact(contact), Integer rows -> {
                Toast.makeText( context: this, rows == 1 ? R.string.deleted : R.string.missing_contact,
                        Toast.LENGTH_SHORT).show();
                reload();
            });
            return true;
        });
        reload();
```

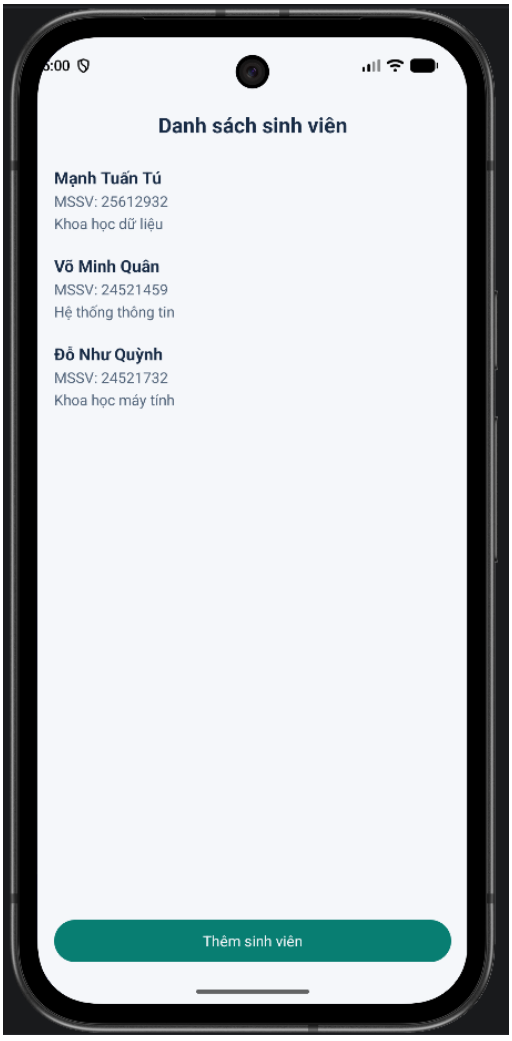
```
    }

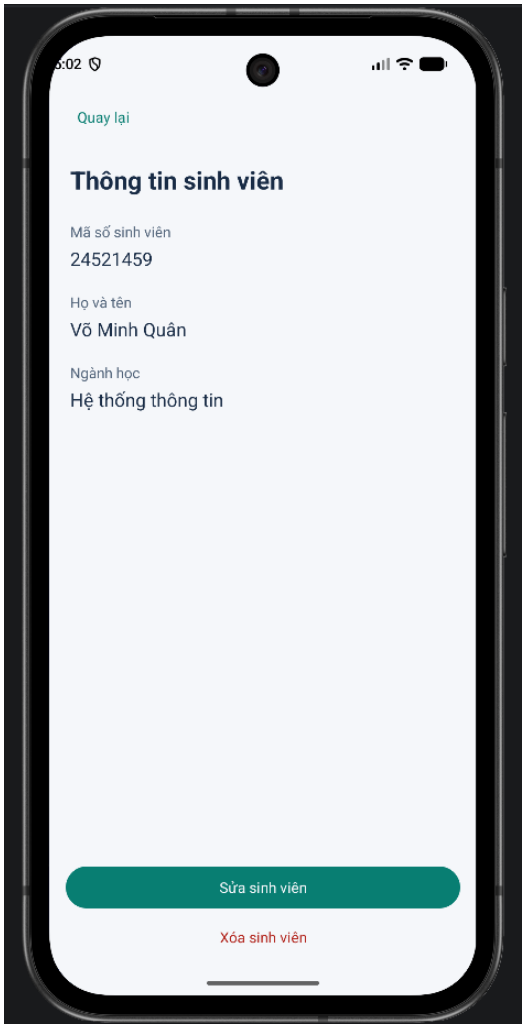
    2 usages
    private void reload() {
        tasks.submit(database::getAllContacts, adapter::replace);
    }

    @Override protected void onDestroy() {
        tasks.close();
        super.onDestroy();
    }
}
```

Bài 2: Quản lý Students

Kết quả:





Student.java

```java
public final class Student {
    2 usages
    private final String id;
    2 usages
    private final String name;
    2 usages
    private final String major;

    4 usages
    public Student(String id, String name) { this(id, name, major: ""); }

    3 usages
    public Student(String id, String name, String major) {
        this.id = id;
        this.name = name;
        this.major = major;
    }
    public String getId() { return id; }
    public String getName() { return name; }
    3 usages
    public String getMajor() { return major; }
}
```

DatabaseHandler.java

```java
public final class DatabaseHandler extends SQLiteOpenHelper {
    1 usage
    public static final String DATABASE_NAME = "studentManager";
    1 usage
    private static final int DATABASE_VERSION = 3;
    6 usages
    private static final String TABLE = "students";

    1 usage
    public DatabaseHandler(Context context) { this(context, DATABASE_NAME); }

    4 usages
    DatabaseHandler(Context context, String databaseName) {
        super(context.getApplicationContext(), databaseName, null, DATABASE_VERSION);
    }

    @Override public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE students (id TEXT PRIMARY KEY NOT NULL, name TEXT NOT NULL, "
                + "major TEXT NOT NULL DEFAULT '')");
        db.insertOrThrow(TABLE, null, values(new Student("24521459", "Võ Minh Quân")));
    }

    11 usages
    @Override public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        if (oldVersion < 2) {
            db.execSQL("CREATE TABLE students_new (id TEXT PRIMARY KEY NOT NULL, name TEXT NOT NULL, "
                    + "major TEXT NOT NULL DEFAULT '')");
            db.execSQL("INSERT INTO students_new (id, name, major) SELECT id, name, major FROM students "
                    + "WHERE id NOT IN ('DEMO01', 'DEMO02')");
            db.execSQL("DROP TABLE students");
            db.execSQL("ALTER TABLE students_new RENAME TO students");
```

```
        } else if (oldVersion < 3) {
            db.execSQL("ALTER TABLE students ADD COLUMN major TEXT NOT NULL DEFAULT ''");
        }
    }

    3 usages
    private ContentValues values(Student student) {
        ContentValues values = new ContentValues();
        values.put("id", student.getId());
        values.put("name", student.getName());
        values.put("major", student.getMajor());
        return values;
    }

    /** Returns false for a duplicate primary key; other SQLite failures are propagated. */
    3 usages
    public boolean addStudent(Student student) {
        try {
            getWritableDatabase().insertOrThrow(TABLE, null, values(student));
            return true;
        } catch (SQLiteConstraintException duplicate) {
            if (getStudent(student.getId()) != null) return false;
            throw duplicate;
        }
    }

    8 usages
    public Student getStudent(String id) {
        try (Cursor cursor = getReadableDatabase().query(TABLE, null, "id = ?",
                new String[]{id}, null, null, null)) {
            return cursor.moveToFirst() ? readStudent(cursor) : null;
        }
    }
```

```
    5 usages
    public List<Student> getAllStudents() {
        List<Student> students = new ArrayList<>();
        try (Cursor cursor = getReadableDatabase().query(TABLE, null, null, null, null,
                null, "name COLLATE NOCASE ASC, id ASC")) {
            while (cursor.moveToNext()) students.add(readStudent(cursor));
        }
        return students;
    }

    3 usages
    public int updateStudent(Student student) {
        return getWritableDatabase().update(TABLE, values(student), "id = ?", new String[]{student.getId()});
    }

    4 usages
    public int deleteStudent(String id) {
        return getWritableDatabase().delete(TABLE, "id = ?", new String[]{id});
    }

    2 usages
    private Student readStudent(Cursor cursor) {
        return new Student(cursor.getString(cursor.getColumnIndexOrThrow("id")),
                cursor.getString(cursor.getColumnIndexOrThrow("name")),
                cursor.getString(cursor.getColumnIndexOrThrow("major")));
    }
}
```

StudentAdapter.java

```
final class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.Holder> {
    2 usages
    private final Consumer<Student> onSelected;
    4 usages
    private final List<Student> students = new ArrayList<>();

    1 usage
    StudentAdapter(Consumer<Student> onSelected) { this.onSelected = onSelected; }

    1 usage
    void replace(List<Student> newStudents) {
        students.clear();
        students.addAll(newStudents);
        notifyDataSetChanged();
    }

    @NonNull @Override public Holder onCreateViewHolder(@NonNull ViewGroup parent, int type) {
        View row = LayoutInflater.from(parent.getContext())
                .inflate(R.layout.item_student, parent, attachToRoot: false);
        return new Holder(row);
    }

    @Override public void onBindViewHolder(@NonNull Holder holder, int position) {
        Student student = students.get(position);
        holder.name.setText(student.getName());
        holder.code.setText("MSSV: {student.getId()}");
        holder.major.setText(student.getMajor());
        holder.major.setVisibility(student.getMajor().isEmpty() ? View.GONE : View.VISIBLE);
        holder.itemView.setOnClickListener( View view -> onSelected.accept(student));
    }
```

```
    @Override public int getItemCount() { return students.size(); }

    4 usages
    static final class Holder extends RecyclerView.ViewHolder {
        2 usages
        final TextView name;
        2 usages
        final TextView code;
        3 usages
        final TextView major;

        1 usage
        Holder(View item) {
            super(item);
            name = item.findViewById(R.id.studentName);
            code = item.findViewById(R.id.studentCode);
            major = item.findViewById(R.id.studentMajor);
        }
    }
}
```

MainActivity.xml

```
public class MainActivity extends DatabaseActivity {
    private StudentAdapter adapter;
    private View empty;

    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_main);
        applyInsets();

        empty = findViewById(R.id.emptyView);
        RecyclerView list = findViewById(R.id.studentList);
        list.setLayoutManager(new LinearLayoutManager(this));
        adapter = new StudentAdapter( Student student -> startActivity(new Intent(this,
                StudentDetailActivity.class).putExtra(EXTRA_STUDENT_ID, student.getId())));
        list.setAdapter(adapter);
        findViewById(R.id.addButton).setOnClickListener( View view ->
                startActivity(new Intent(this, StudentEditorActivity.class)));
    }

    @Override protected void onResume() {
        super.onResume();
        tasks.<List<Student>>submit(database::getAllStudents, students -> {
            adapter.replace(students);
            empty.setVisibility(students.isEmpty() ? View.VISIBLE : View.GONE);
        });
    }
}
```

StudentDetailActivity

```java
public class StudentDetailActivity extends DatabaseActivity {
    6 usages
    private String studentId;
    4 usages
    private Student current;

    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_student_detail);
        applyInsets();
        studentId = getIntent().getStringExtra(EXTRA_STUDENT_ID);
        if (studentId == null) { finish(); return; }
        findViewById(R.id.backButton).setOnClickListener( View view -> finish());
        findViewById(R.id.editButton).setEnabled(false);
        findViewById(R.id.deleteButton).setEnabled(false);
        findViewById(R.id.editButton).setOnClickListener( View view ->
                startActivity(new Intent( packageContext: this, StudentEditorActivity.class).putExtra(EXTRA_STUDENT_ID, studentId)));
        findViewById(R.id.deleteButton).setOnClickListener( View view -> confirmDelete());
    }

    @Override protected void onResume() {
        super.onResume();
        if (studentId == null) return;
        tasks.submit(() -> database.getStudent(studentId), student -> {
            if (student == null) { toast("Sinh viên không còn tồn tại."); finish(); return; }
            current = student;
            setText(R.id.detailName, student.getName());
            setText(R.id.detailCode, student.getId());
            setText(R.id.detailMajor, student.getMajor().isEmpty()
                    ? "Chưa chọn ngành" : student.getMajor());
            findViewById(R.id.editButton).setEnabled(true);
            findViewById(R.id.deleteButton).setEnabled(true);
        });
    }
```

```java
    private void setText(int id, String text) { ((TextView) findViewById(id)).setText(text); }

    1 usage
    private void confirmDelete() {
        if (current == null) return;
        new MaterialAlertDialogBuilder( context: this).setTitle("Xóa sinh viên?")
                .setMessage("Xóa {current.getName()} ({current.getId()})?")
                .setNegativeButton("Hủy", listener: null)
                .setPositiveButton("Xóa sinh viên", ( DialogInterface dialog, int which) -> {
                    findViewById(R.id.deleteButton).setEnabled(false);
                    tasks.submit(() -> database.deleteStudent(studentId), rows -> {
                        toast(rows == 1 ? "Đã xóa sinh viên." : "Sinh viên không còn tồn tại.");
                        finish();
                    }, () -> findViewById(R.id.deleteButton).setEnabled(true));
                }).show();
    }
}
```

PHỤ LỤC

Mã nguồn của lab:
https://github.com/NT118-R12-24521459-Lab/Lab03